When Training Data Are Gone: Selective Integration of Pretrained Intrusion Detectors

***Abstract*—Effective intrusion detection benefits from diverse attack data, yet historical source records may no longer be accessible due to privacy and retention constraints, preventing further collaborative training. In this paper, we study selective integration of authorized pretrained detectors, aiming to choose a fixed detector set under a hard deployment budget while preserving diverse source attack-class knowledge and maintaining predictive quality on current deployment data. We present SILOSTITCH, which treats source coverage and current predictive behavior as ordered objectives rather than combining them into a single score. SILOSTITCH aligns heterogeneous detector outputs, uses held-out deployment records to assess current behavior, exactly maximizes weighted source class coverage under a logical byte budget via dynamic programming, and then refines the selected set to improve current prediction without reducing coverage. A shared stacker integrates the selected outputs, with optional Laplace perturbation for score uploads. Theoretically, we establish that calibration remains valid after adaptive selection and that the primary coverage problem is solved exactly, while perturbed scores satisfy label-conditional local differential privacy. Across four cybersecurity datasets with 60+ candidate detectors per task, SILOSTITCH reduces communication by approximately half under a 50% selection-cost budget while preserving maximum weighted source coverage and achieving detection performance comparable to the full detector pool.**

I. INTRODUCTION

Effective intrusion detection relies on training data that capture diverse traffic conditions and attack behaviors. However, data collected by a single organization typically reflect only its own operational environment, limiting the diversity of attack patterns available for training. Collaborative learning, particularly Federated Learning (FL), can mitigate such data silos by learning from distributed records without pooling them centrally [1]–[4]. Yet these approaches still require contributors to retain their local training records and participate in additional model optimization. In practice, historical network logs may become inaccessible because they contain sensitive information, such as internal network topologies and user identities [5], [6], and may need to be deleted under privacy and retention requirements such as the GDPR [7]. When these training records are no longer available but pretrained detectors remain authorized for reuse, integrating existing detectors provides an alternative way to exploit complementary source knowledge without revisiting the original data.

Pretrained-model reuse is therefore particularly suitable for this setting [8], [9]. However, existing approaches either select or aggregate pretrained models without explicitly accounting for heterogeneous deployment costs [10], [11], or perform cost-aware dynamic detector selection rather than choosing a single fixed detector set for deployment [12]. This gap motivates our research question: *How can a coordinator selectively integrate a fixed set of pretrained intrusion detectors under a deployment budget while preserving diverse source attack-class coverage and maintaining detection performance on the current deployment data?*

Answering this question presents three challenges. First, detectors contributed by different parties may have different label spaces and support different attack classes, making their outputs difficult to compare and integrate directly. Second, during integration, preserving source attack knowledge and maintaining current detection performance are not always aligned. A detector that performs well on current deployment data may contribute little new class knowledge, whereas a detector with unique class coverage may no longer perform well after distribution shifts. Therefore, the coordinator must preserve unique source coverage without filling the limited budget with detectors that are currently weak or redundant. Third, detector selection must satisfy the deployment budget while remaining statistically reliable and privacy-aware. Because calibration data are used to choose detectors, their performance estimates must remain valid after selection, while uploaded prediction scores may still reveal sensitive information.

To address these challenges, we present SILOSTITCH, a framework for selective integration of frozen intrusion detectors under deployment budgets. The key idea is to prioritize source class coverage over current predictive performance: SILOSTITCH first maximizes weighted source class coverage under the deployment budget, and then improves current predictive behavior without reducing that coverage. This priority separates historical class coverage from current predictive quality: source class counts indicate which attack classes were represented during training, whereas held-out deployment records indicate how well the pretrained detectors perform after distribution shifts [13]. To realize this design, SILOSTITCH first aligns heterogeneous detector outputs to a common label space and uses held-out deployment records to assess their current predictive behavior. It then exactly maximizes weighted source class coverage under the budget through dynamic programming, refines the selected set without sacrificing coverage, and integrates the selected outputs with a shared stacker [14].

Additionally, we theoretically prove that calibration remains valid after adaptive detector selection and that the primary coverage problem is solved exactly. Because avoiding raw-data sharing alone does not eliminate privacy risks from model outputs [15], SILOSTITCH also supports optional Laplace perturbation before score upload. The perturbed scores provide

record-level, label-conditional local differential privacy, while this guarantee does not extend to public labels or contributed detector parameters.

We evaluate SILOSTITCH on four intrusion detection datasets with over 60 candidate detectors per task. At a 50% selection cost budget, SILOSTITCH maximizes weighted source class coverage while reducing total logical communication by 49.3% to 50.4%, demonstrating that diverse source knowledge can be retained at roughly half the communication cost. Importantly, these savings incur only a small loss in detection quality: across all four datasets, the macro F1 gap from the full detector pool is limited to 0.028. Consistent with the exactness guarantee, the primary solver attains the same coverage as exhaustive search in all comparison instances. In summary, we make four contributions.

- We formulate budget-constrained selective integration of pretrained intrusion detectors when historical source records are unavailable and introduce an ordered objective that prioritizes source class coverage over current predictive behavior.
- We develop SILOSTITCH to align heterogeneous detectors, exactly maximize weighted source coverage under the budget, refine the selected set without reducing coverage, and integrate their outputs with a shared stacker.
- We prove selection-valid calibration, exact primary coverage optimization, and local refinement stability, and provide privacy and mixture-shift guarantees.
- We evaluate SILOSTITCH on four intrusion detection tasks, showing that a 50% selection-cost budget roughly halves communication while preserving maximum source coverage and near-full-pool detection quality.

II. BACKGROUND AND RELATED WORK

We first present and review existing collaborative training and model integration methods, and then introduce the notion of differential privacy.

Retraining and Adaptation of Intrusion Detector. When participants retain local training data and can rerun optimization, federated learning coordinates their parameter updates without transferring raw data [1]–[3]. Within intrusion detection, federated methods adapt this process to non-IID traffic and heterogeneous local learners [6], [19]. Recent NIDS research also redesigns the training pipeline to withstand adverse data conditions. For example, RAPIER handles low quality labels for encrypted malicious traffic [20], Rosetta uses TCP aware augmentation to improve robustness across network environments [21], and CoLD denoises labels before fitting a classifier [22]. These approaches assume that participants can revisit training, whereas in practice the historical training data may not be availabl after the detector is trained and deployed.

Pretrained Model Integration. When source training data are no longer available but their trained models remain accessible, pretrained model integration is the closest line of work. ZooD ranks a heterogeneous model zoo by cross domain transferability and aggregates its top K models [10], while other approaches combine pretrained models without accessing their source data [9], [11]. However, they focus on improving integration utility but do not consider to comply with the deployment budget.

Output Level Ensembles and Cost-Aware Selection. Once detector outputs are available, stacked generalization can train a downstream classifier over them [14]. SIRAJ aggregates reports from heterogeneous threat intelligence sources by pretraining an embedding and finetuning downstream predictors [16], while FedMStacking trains a global meta classifier from heterogeneous local predictions when labels are missing [18]. To account for inference cost, SPIREL learns a per item policy that trades off model invocations against classification utility [12]. Although these approaches combine outputs or control per item cost, none simultaneously screens detectors on current data and returns one fixed subset under a hard aggregate budget. By contrast, SILOSTITCH makes this subset decision before fitting a shared classifier over the selected outputs.

Table I compares the closest methods across eight capabilities that define our setting. For model integration, we record whether each method accepts pretrained, heterogeneous models and stacks their outputs. For subset selection, we record whether each method screens detectors on current data and returns one fixed set under a hard budget while treating available class support as a strict priority. The final capability records whether each method randomizes predictions locally before upload.

When a method offers related but non equivalent functionality, we assign a partial mark. Because SIRAJ learns from scanner reports, it does not accept the pretrained model objects considered here. ZooD instead ranks models using scores from held-out source domains and aggregates a fixed set of its top K models, enforcing a cardinality limit rather than an aggregate cost limit. SPIREL incorporates cost into a per item policy but does not return one fixed detector set. PrivateKT aggregates randomized predictions rather than fitting a fixed score stacker. Although FedMStacking addresses missing labels, it does not treat class support as a strict selection priority. Together, these gaps motivate the ordered selection problem that we formalize next.

Local Differential Privacy. Prediction outputs can reveal information about their inputs, so a privacy guarantee must specify both where randomization occurs and which fields it protects. For example, PrivateKT perturbs locally produced predictions before uploading them for knowledge transfer [17], whereas differentially private federated trees protect the statistics exchanged while training a new model [23]. Our setting differs because SILOSTITCH uploads scores from selected detectors to fit a shared classifier while treating labels as public. We therefore specialize the local model of Duchi et al. [24] to this release.

Definition II.1 (Label conditional local differential privacy)**.** For a fixed public label y, a client applies a randomized mechanism $\mathcal{M}_y$ to the feature vector x. The upload $\mathcal{R}(x, y) =$

Table I
COMPARISON WITH RELATED METHODS. ✓DENOTES DIRECT SUPPORT, ●RELATED SUPPORT, AND ○NO SUPPORT.

Method	Model Integration			Subset Selection				Privacy
	pretrained Models	Heterogeneous	Score Stacking	Current Screening	Fixed Set	Hard Budget	Class Priority	Score LDP
SIRAJ [16]	○	✓	●	○	○	○	○	○
PrivateKT [17]	○	○	●	○	○	○	○	✓
SPIREL [12]	✓	✓	●	○	●	●	○	○
ZooD [10]	✓	✓	●	○	✓	●	○	○
FedMStacking [18]	○	✓	✓	○	○	○	○	○
SILOSTITCH	✓	✓	✓	✓	✓	✓	✓	✓

$(\mathcal{M}_y(x), y)$ satisfies label conditional $(\epsilon, 0)$-local differential privacy if, for every fixed y, every pair x, x', and every measurable output set $\mathcal{O}$,

$$\Pr[\mathcal{M}_y(x) \in \mathcal{O}] \leq e^{\epsilon} \Pr[\mathcal{M}_y(x') \in \mathcal{O}]. \tag{1}$$

This definition compares records with the same public label and protects the feature derived portion of the upload, and the label itself remains public.

To instantiate Definition II.1, let the deterministic score map g have replacement sensitivity

$$\Delta_1(g) = \sup_{x,x'} \|g(x) - g(x')\|_1. \tag{2}$$

Adding independent noise $\eta_k \sim \mathrm{Lap}(0, \Delta_1(g)/\epsilon)$ to every coordinate yields a mechanism that satisfies Definition II.1 [25]. If one record releases blocks with parameters $\epsilon_1, \ldots, \epsilon_s$, sequential composition gives the parameter $\sum_{i=1}^{s} \epsilon_i$. Under one record replacement, releases derived from unchanged records do not increase this privacy parameter.

In SILOSTITCH, this mechanism applies to the selected detector score blocks. Theorem V.1 establishes label conditional local privacy for the resulting score upload under the public label adjacency relation.

III. PROBLEM STATEMENT

Post-training integration must select among pretrained detectors even when their source records are no longer available. We formalize this setting, define the ordered selection objective, and state the execution and information boundaries used by the design and analysis.

A. System Model

After local fitting ends, J clients expose a pool of M pretrained candidate detectors. Each client contributes one candidate in our experiments, although SILOSTITCH allows J and M to vary independently. For candidate i, let $p_i(x) \in \mathbb{R}^{h_i}$ denote its native probability output. Because candidates may use incompatible label coordinates, the coordinator validates a hard semantic map

$$A_i \in \{0,1\}^{(C+1)\times h_i}, \qquad \mathbf{1}^\top A_i = \mathbf{1}^\top, \tag{3}$$

and forms

$$q_i(x) = A_i p_i(x) \in \Delta^C, \tag{4}$$

where Δ^C is the probability simplex with $C+1$ coordinates. We align the first C coordinates with a pretrained target ontology and reserve the last coordinate, denoted by $\perp$, for unsupported outputs. Every later Brier calculation therefore retains, rather than silently discarding, probability mass assigned to $\perp$.

Before selection, the coordinator holds a labeled calibration set $\mathcal{D}^{\mathrm{cal}}$ from the current deployment domain. We keep this set disjoint from the records used for candidate fitting, stacker fitting, and sealed testing. The coordinator also freezes every candidate, semantic map, and source side class count before evaluating the pool. Test labels remain sealed until both the selected set and the stacker are fixed, so they cannot affect calibration, cost, or selection.

Selection consumes communication in both model delivery and score upload. To enforce one hard budget across these steps, SILOSTITCH assigns each selected candidate a logical cost. Let K_i be candidate i's serialized size, let B_t be the number of stacker fit records, and let d be the logical byte width of one probability element. For a selected set S, define

$$C_{\mathrm{sel}}(S) = \sum_{i \in S} b_i, \qquad b_i = JK_i + dB_t(C+1). \tag{5}$$

The first term accounts for delivering a selected candidate to J clients, whereas the second accounts for uploading one aligned probability block per stacker fit record. Given a budget fraction β, we set

$$B = \left\lfloor \beta \sum_{i=1}^{M} b_i \right\rfloor. \tag{6}$$

The denominator includes every candidate, including candidates that may later fail calibration. For complete logical communication, we additionally count the initial candidate upload and the stacker label upload:

$$C_{\mathrm{total}}(S) = \sum_{i=1}^{M} K_i + C_{\mathrm{sel}}(S) + dB_t. \tag{7}$$

We use C_{sel} to enforce the selection budget and C_{total} to compare the complete communication of selected and full pool configurations.

B. Detector Selection Objective

Alignment makes detector outputs comparable, but it does not determine which detectors should survive the budget. We next define the ordered objective that governs this choice.

Ranking only by average current utility may discard the sole affordable candidate that supports an important class. Ranking only by source side support can instead retain candidates whose current probabilities are weak or redundant. A single weighted sum does not resolve this conflict because enough secondary gain can compensate for a positive coverage loss. SILOSTITCH therefore gives weighted source side coverage strict priority and compares current behavior only among sets that attain the same primary value.

With that priority fixed, SILOSTITCH targets

$$\operatorname{lexmax}_{\varnothing \neq S \subseteq \widehat{\mathcal{R}}} \left(C_{\mathrm{sem}}(S), \Phi(S)\right) \quad \text{s.t.} \quad \sum_{i \in S} b_i \leq B. \tag{8}$$

Here, $\widehat{\mathcal{R}}$ contains the candidates that pass calibration, C_{sem} measures weighted source support, and Φ summarizes current predictive scores. The primary solver computes the first coordinate exactly. The deployed refinement then improves the secondary coordinate only through coverage neutral local operations, and it does not claim a globally optimal secondary value. Only the optional small pool exhaustive solver computes the complete lexicographic optimum.

C. Execution and Analysis Model

The guarantees depend on which artifacts the coordinator observes and where randomization occurs. We therefore make the execution and privacy models explicit.

We evaluate the workflow in a centralized simulation, with Figure 1 tracing the protocol's logical data dependencies. In a client side realization, each client perturbs its selected probability blocks with fresh private Laplace randomness before upload. The optional sensitivity mode samples the same distribution centrally only to measure its utility effect.

The privacy analysis treats labels as public and protects the feature dependent information conveyed by the randomized score uploads used to fit the stacker. It assumes immutable candidate detectors, structurally validated semantic maps, honest source support reports, and fresh client side randomness. The resulting guarantee is label conditional and applies to the score upload, not to the contributed detector parameters.

IV. DESIGN OF SILOSTITCH

SILOSTITCH addresses the three design challenges in sequence: it makes heterogeneous outputs comparable, selects detectors without relaxing the primary coverage target, and learns from only the selected outputs. This section follows that sequence from calibration through shared stacking.

A. Selection and Stacking Workflow

Detector selection and stacker fitting require separate data so that stacker fit records cannot influence which detectors are chosen. As Figure 1 shows, the coordinator uses only calibration outputs to determine eligibility and the selected set. After the coordinator freezes that set, clients evaluate the independent stacker fit records and upload only the selected aligned blocks, optionally perturbed, together with their labels. Algorithm 1 summarizes the complete procedure between client and server, and the following subsections define CALIBRATE, COVER, and REFINE in detail.

Algorithm 1 SILOSTITCH Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^{M}$, $\mathcal{D}^{\mathrm{cal}}$, $\{\mathcal{D}_j^{\mathrm{stk}}\}_{j=1}^{J}$, B, $\epsilon_b \in (0, \infty]$
Ensure: $\widehat{S}$, $\widehat{f}$

```
Clients → Server: {f_i, A_i, H_i, K_i}_{i=1}^M
(R̂, Γ, w, b, Φ) ← CALIBRATE(D^cal)
(W*, S_0) ← COVER(R̂, Γ, w, b, B)
Ŝ ← REFINE(S_0, W*, Φ, B)
Server → Clients: {f_i, A_i}_{i∈Ŝ}
for j = 1, ..., J in parallel do
    Z_j ← concat_{i∈Ŝ} q_i(X_j^stk)
    (Ξ_j)_{t,i,k} ~iid Lap(0, 2/ε_b)
    Z̃_j ← Z_j + Ξ_j
    Client j → Server: (Z̃_j, Y_j^stk)
end for
f̂ ← FIT(⋃_j (Z̃_j, Y_j^stk))
return (Ŝ, f̂)
```

B. Probability Alignment and Brier Calibration

The selector needs comparable, bounded scores from detectors with different native outputs. We obtain these scores by evaluating the aligned probability vectors with a full vector Brier score.

For calibration record (x_t, y_t), let $e_{y_t} \in \mathbb{R}^{C+1}$ be the one hot label vector with a zero $\perp$ coordinate. We evaluate each candidate with the complete aligned output and define

$$\ell_{i,t} = \frac{1}{2} \left\| q_i(x_t) - e_{y_t} \right\|_2^2, \qquad g_{i,t} = 1 - \ell_{i,t} \in [0, 1]. \tag{9}$$

Thus, $g_{i,t}$ is one full vector Brier utility rather than a collection of coordinate wise scores. Including the $\perp$ coordinate ensures that unsupported probability mass reduces this utility.

Eligibility screening. An empirical mean alone can admit a candidate whose current risk is too high or too uncertain. Using N calibration records and failure level $\delta \in (0, 1)$, we compute

$$\widehat{\mu}_i = \frac{1}{N} \sum_{t=1}^{N} g_{i,t}, \qquad r = \sqrt{\frac{\log(4M/\delta)}{2N}}, \tag{10}$$

and admit candidate i only if

$$R_i^{+} = 1 - \widehat{\mu}_i + r \leq \tau_L. \tag{11}$$

The gate defines the calibration eligible pool

$$\widehat{\mathcal{R}} = \{i : R_i^{+} \leq \tau_L\}. \tag{12}$$

Eligibility therefore depends on an upper confidence bound on current risk, not on empirical utility alone.

Classwise scores. Average eligibility does not reveal whether a candidate remains useful for a particular class. We retain the class-specific scores needed by secondary refinement and the

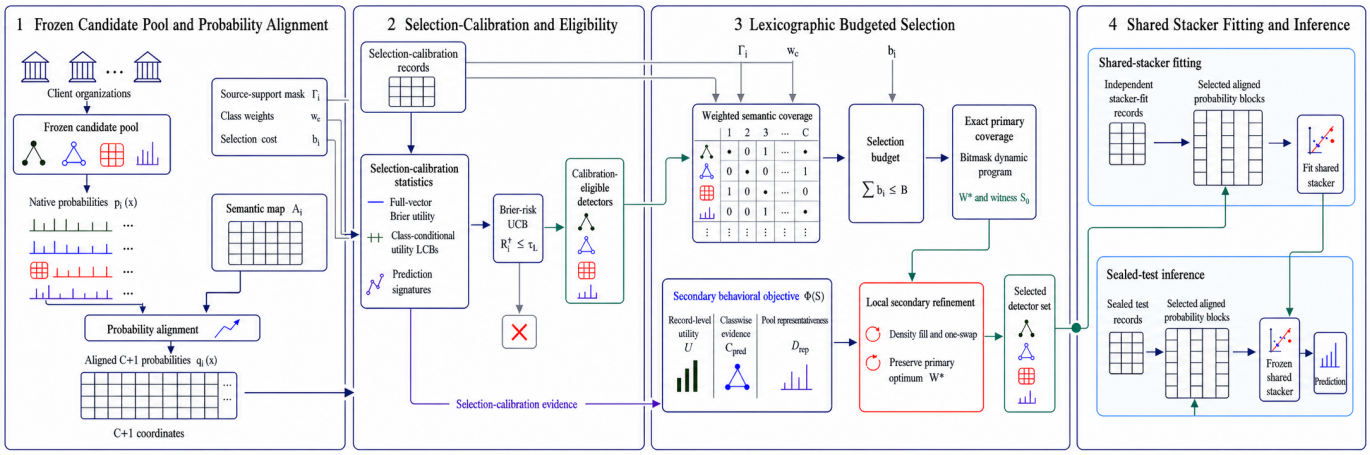

Figure 1. SILOSTITCH first freezes and probability aligns the candidate detector pool. It then uses selection calibration records to form the calibration eligible pool, obtains the exact maximum weighted semantic coverage under the selection budget, and performs coverage neutral local secondary refinement. Finally, selected aligned probability blocks support independent shared stacker fitting and sealed test inference. Arrows denote the workflow's logical data dependencies.

later coverage certificate. For class c, let $N_c = \sum_t \mathbf{1}\{y_t = c\}$ and

$$\widehat{\mu}_{i,c} = \frac{1}{N_c} \sum_{t:y_t=c} g_{i,t}. \tag{13}$$

Because SILOSTITCH requires every calibration class to occur, $N_c > 0$. We then compute the class conditional radius and lower utility as

$$r_c = \sqrt{\frac{\log(4MC/\delta)}{2N_c}}, \qquad a_{i,c} = [\widehat{\mu}_{i,c} - r_c]_+, \tag{14}$$

where $[z]_+ = \max\{0, z\}$. The secondary objective and effective coverage certificate use $a_{i,c}$ as a conservative class-specific score.

Behavioral similarity. Eligibility alone does not prevent two candidates from behaving almost identically. SILOSTITCH measures this redundancy on a deterministically selected, class stratified anchor set $\mathcal{A}$ and forms

$$z_i = \mathrm{vec}\left([q_i(x_t)]_{t\in\mathcal{A}}\right), \qquad \kappa_{i,k} = \left[\frac{\langle z_i, z_k\rangle}{\|z_i\|_2\|z_k\|_2}\right]_0^1. \tag{15}$$

Here, $[\cdot]_0^1$ clips numerical roundoff into $[0, 1]$. We use $\kappa_{i,k}$ in a facility location term that measures representativeness, not as a pairwise diversity score.

C. Coverage Prioritized Detector Selection

Calibration measures current behavior, but it cannot preserve the range of source side class support by itself. We therefore make weighted semantic coverage the primary budgeted objective.

Weighted semantic coverage. For candidate i and target class c, let $H_{i,c}$ be the accepted source side support count after semantic mapping. After fixing $n_{\min} > 0$, define

$$\Gamma_{i,c} = \mathbf{1}\{H_{i,c} \geq n_{\min}\}. \tag{16}$$

If $H_c = \sum_i H_{i,c} > 0$, we define

$$w_c = \frac{H_c^{-1/2}}{\sum_{d=1}^{C} H_d^{-1/2}}, \qquad \sum_{c=1}^{C} w_c = 1. \tag{17}$$

Classes with less aggregate source support consequently receive greater weight. For any selected set S,

$$C_{\mathrm{sem}}(S) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1\}. \tag{18}$$

Because the $\perp$ coordinate represents unsupported outputs, we exclude it from this coverage. Accordingly, C_{sem} measures the range of declared source support rather than current predictive accuracy.

Secondary behavioral objective. Once the primary coverage value is fixed, current scores can distinguish among coverage equivalent sets. With $h_\tau(z) = 1 - e^{-z/\tau}$, define

$$\begin{aligned} U(S) &= \frac{1}{N}\sum_{t=1}^{N} h_{\tau_U}\left(\sum_{i\in S} g_{i,t}\right), \\ C_{\mathrm{pred}}(S) &= \frac{1}{C}\sum_{c=1}^{C} h_{\tau_C}\left(\sum_{i\in S} a_{i,c}\right), \\ D_{\mathrm{rep}}(S) &= \frac{1}{|\widehat{\mathcal{R}}|}\sum_{k\in\widehat{\mathcal{R}}} \max_{i\in S} \kappa_{i,k}. \end{aligned} \tag{19}$$

The term U summarizes record level utility, C_{pred} summarizes conservative classwise scores, and D_{rep} rewards behavior that represents the eligible pool. The saturating functions impose diminishing returns so that repeated scores cannot dominate the secondary comparison without bound. Here and throughout the analysis, we set $\max_{i\in\varnothing} \kappa_{i,k} = 0$. We average the classes uniformly in C_{pred}, whereas C_{sem} uses source support weights. For nonnegative weights that sum to one, the secondary objective is

$$\Phi(S) = \lambda_U U(S) + \lambda_C C_{\mathrm{pred}}(S) + \lambda_D D_{\mathrm{rep}}(S). \tag{20}$$

SILOSTITCH uses this score only after fixing semantic coverage, so no increase in Φ can compensate for a decrease in C_{sem}.

Budgeted selection procedure. The hard budget induces the feasible family and exact primary target

$$\mathcal{F}_B = \left\{ \varnothing \neq S \subseteq \widehat{\mathcal{R}} : \sum_{i \in S} b_i \leq B \right\}, \qquad W^\star = \max_{S \in \mathcal{F}_B} C_{\mathrm{sem}}(S). \tag{21}$$

We encode every Γ_i as a C-bit mask and use a sparse dynamic program to compute $W^\star$ and recover a least cost witness. Starting from this exact primary witness, we spend remaining budget through density-based fill, accepting the chosen candidate only while its absolute gain exceeds $\varepsilon_{\mathrm{loc}}$. We then accept a single swap replacement only when it preserves $W^\star$ and improves Φ by more than the same fixed tolerance. These operations refine the secondary objective locally without weakening the exact primary guarantee.

For a matched comparison, we also apply the same fill–one swap procedure without semantic priority. If that no semantic result already attains $W^\star$, SILOSTITCH returns the same set unchanged. Otherwise, it refines the exact coverage witness and, when available, a coverage optimal singleton seed, then returns the locally refined result with the larger Φ value. The two variants thus coincide when the semantic constraint is inactive and differ only when semantic priority changes the refinement path.

D. Shared Stacker Learning

After the selector freezes $\widehat{S}$, only the selected aligned outputs enter the shared representation:

$$\phi_{\widehat{S}}(x) = \mathrm{concat}_{i \in \widehat{S}}\, q_i(x) \in \mathbb{R}^{|\widehat{S}|(C+1)}. \tag{22}$$

Local score perturbation. Selected score blocks may reveal record level information even though source records are not shared. When local score protection is enabled, client j independently perturbs every coordinate of each selected block:

$$\widetilde{q}_i(x_t) = q_i(x_t) + \eta_{i,t}, \qquad \eta_{i,t,k} \overset{\mathrm{iid}}{\sim} \mathrm{Lap}\left(0, \frac{2}{\epsilon_b}\right). \tag{23}$$

The resulting perturbed representation is

$$\widetilde{\phi}_{\widehat{S}}(x) = \mathrm{concat}_{i \in \widehat{S}}\, \widetilde{q}_i(x). \tag{24}$$

Shared stacker fitting. The independent stacker fit partition $\mathcal{D}^{\mathrm{stk}}$ supplies the labels and selected score features used to train

$$\widehat{f}_{\widehat{S}} = \mathsf{Fit}_{\mathrm{global}}\left(\{ (\widetilde{\phi}_{\widehat{S}}(x_t), y_t) : (x_t, y_t) \in \mathcal{D}^{\mathrm{stk}} \} \right). \tag{25}$$

For the nonprivate configuration, we set $\widetilde{\phi}_{\widehat{S}} = \phi_{\widehat{S}}$. Both configurations use only the columns indexed by $\widehat{S}$, so unselected blocks cannot affect the learned classifier.

The centralized sensitivity experiment samples from Eq. 23 after selection to isolate the utility effect of noise. In a distributed realization, each client samples the same noise privately. During sealed test inference, SILOSTITCH preserves the selected layout and leaves every contributed detector unchanged.

V. THEORETICAL ANALYSIS

The analysis follows the guarantee chain created by the design: privacy holds at score upload, calibration remains valid under adaptive selection, the primary solver is exact, and the secondary refinement has a local certificate. We then bound how client mixture shift changes the interpretation of the pretrained set's current scores.

A. Local Privacy of Score Uploads

Because the coordinator receives selected score blocks, the first question is what the client side randomization protects. We answer it under the execution boundary defined in Section III-C.

We fix the selected set before observing any stacker fit record and treat each record label as public. Accordingly, (x, y) and (x', y) are adjacent only when their feature vectors differ. Because Algorithm 1 uploads y without perturbation, we use this label fixed relation. For a batched upload, adjacent batches have the same size, row alignment, and label sequence, and they differ in exactly one feature vector. Each client samples noise independently across records, selected detector blocks, and output coordinates.

Theorem V.1 (Label conditional local score privacy)**.** Suppose that the selected detectors and their semantic maps are fixed and that $0 < \epsilon_b < \infty$ and each client uses fresh private randomness in Eq. 23. Then one released detector block is $(\epsilon_b, 0)$-locally differentially private under the preceding adjacency relation. If $s = |\widehat{S}|$ blocks are released for the same record, their joint release is $(s\epsilon_b, 0)$-locally differentially private. Under one record replacement, releasing the entire client batch has the same record level guarantee. More generally, assigning parameter ϵ_i to block i gives $(\sum_{i \in \widehat{S}} \epsilon_i, 0)$-local differential privacy.

Proof. Because $q_i(x)$ and $q_i(x')$ are probability vectors, $\|q_i(x) - q_i(x')\|_1 \leq 2$. Therefore, coordinatewise Laplace noise with scale $2/\epsilon_b$ bounds the density ratio of one released block by e^{ϵ_b}. Sequential composition over the s selected blocks gives $e^{s\epsilon_b}$. Because different rows use independent randomization and neighboring batches differ in one row, parallel composition across rows leaves this bound unchanged. Appending the unchanged public labels does not alter the density ratio. □

Assigning $\epsilon_b = \epsilon/s$ to each block changes the noise scale to $2s/\epsilon$ and yields a total per record score budget ϵ. For a fixed total budget, selecting fewer blocks also reduces the noise scale required by this composition rule. This guarantee is label conditional and covers the randomized score upload. Privacy parameters compose across repeated upload rounds.

B. Calibration under Heterogeneous Clients

The privacy result concerns the stacker fit partition, whereas detector eligibility depends on a separate calibration partition. We next show that the calibration scores remain valid after the coordinator uses it to select a detector set.

The records in $\mathcal{D}^{\mathrm{cal}}$ may represent different clients and classes, so an identically distributed population model would obscure the mixture being certified. We condition on both the observed client–label design $\mathcal{I} = \{(j_t, y_t)\}_{t=1}^{N}$ and the candidate pool $\mathcal{Q}$ pretrained before calibration. We assume conditional independence given $(\mathcal{Q}, \mathcal{I})$ while allowing the conditional distribution to differ across records. Define

$$\mu_i = \frac{1}{N}\sum_{t=1}^{N} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}], \qquad R_i = 1 - \mu_i, \tag{26}$$

and

$$\mu_{i,c} = \frac{1}{N_c}\sum_{t:y_t=c} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}]. \tag{27}$$

These means describe the record weighted calibration mixture, not equal client or worst client performance.

Theorem V.2 (Simultaneous calibration)**.** Under the preceding assumptions, with probability at least $1 - \delta$, the following inequalities hold simultaneously for every candidate i and class c:

$$R_i \le R_i^{+} \le R_i + 2r, \tag{28}$$

$$\max\{0, \mu_{i,c} - 2r_c\} \le a_{i,c} \le \mu_{i,c}. \tag{29}$$

Consequently, the inequalities remain valid for every candidate in any set selected adaptively from the calibration statistics.

Proof. Conditioning on $(\mathcal{Q}, \mathcal{I})$, Hoeffding's inequality gives

$$\Pr\{|\widehat{\mu}_i - \mu_i| > r \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2Nr^2} = \frac{\delta}{2M}.$$

Therefore, a union bound over M candidates spends at most $\delta/2$. Likewise,

$$\Pr\{|\widehat{\mu}_{i,c} - \mu_{i,c}| > r_c \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2N_c r_c^2} = \frac{\delta}{2MC},$$

so a second union bound over MC pairs spends at most $\delta/2$. The intersection has probability at least $1 - \delta$, and it does not require different candidates evaluated on the same record to be independent.

On this event, $1 - \widehat{\mu}_i + r$ lies between R_i and $R_i + 2r$. Moreover, $\widehat{\mu}_{i,c} - r_c$ lies between $\mu_{i,c} - 2r_c$ and $\mu_{i,c}$. Applying $[\cdot]_+$ proves the class inequalities. Because the event covers all $M + MC$ coordinates before selection, taking a data dependent subset requires no additional union bound over 2^M sets. □

The simultaneous event also constrains how calibration screening can change the best achievable primary coverage. For any pool $\mathcal{E}$, let $W(\mathcal{E})$ be the largest feasible semantic coverage available from that pool, with value zero if no nonempty set is feasible. On the event in Theorem V.2, define

$$\mathcal{R}^{-} = \{i : R_i \le \tau_L - 2r\}, \qquad \mathcal{R}^{0} = \{i : R_i \le \tau_L\}. \tag{30}$$

Then,

$$\mathcal{R}^{-} \subseteq \widehat{\mathcal{R}} \subseteq \mathcal{R}^{0}, \qquad W(\mathcal{R}^{-}) \le W(\widehat{\mathcal{R}}) \le W(\mathcal{R}^{0}). \tag{31}$$

The left inclusion follows from $R_i^{+} \le R_i + 2r$, whereas the right inclusion follows from $R_i \le R_i^{+}$. Hence, calibration preserves the conditional calibration mixture primary value whenever a primary optimal witness over $\mathcal{R}^0$ lies entirely in $\mathcal{R}^{-}$.

With $\tau_L = 1$, the upper confidence gate is a finite sample admissibility screen over normalized Brier risk. We therefore refer to $\widehat{\mathcal{R}}$ as the calibration eligible pool.

C. Exact Primary Coverage under a Hard Budget

Once calibration fixes the eligible pool, the remaining primary challenge is combinatorial: the budget may prevent simultaneous coverage of all classes. We characterize this problem and the exact parameterized solver used by SILOSTITCH.

Theorem V.3 (Primary hardness and exact parameterized solver)**.** Computing $W^{\star}$ is NP hard, even when every candidate is eligible, every cost is one, and all class weights are uniform. Nevertheless, the bitmask dynamic program returns a nonempty feasible set S_0 with

$$C_{\mathrm{sem}}(S_0) = W^{\star}. \tag{32}$$

It stores at most 2^C masks and examines at most $|\widehat{\mathcal{R}}|2^C$ state transitions. Thus, its primary stage is fixed parameter tractable in C.

Proof. For hardness, take a Maximum Coverage instance with universe $\{1, \ldots, C\}$, subsets $\Gamma_1, \ldots, \Gamma_M$, and cardinality limit k. Set $b_i = 1$, $B = k$, and $w_c = 1/C$. Maximizing C_{sem} then solves that instance up to the constant factor $1/C$.

For exactness, after processing the first t eligible candidates, associate each reachable union mask h with its least cost witness. Omitting candidate t preserves an earlier state, whereas including it extends an earlier mask to $h \vee \Gamma_t$. Every subset makes exactly one of these choices. Moreover, a lower cost witness for the same mask dominates every higher cost witness under all future union operations. Induction therefore shows that the final table contains the least cost witness for every reachable mask. Scanning the budget feasible masks returns exactly $W^{\star}$.

A C-bit mask has at most 2^C values, and each eligible candidate extends each current mask at most once. When all feasible masks have value zero, the solver returns the least cost feasible singleton, which enforces the nonempty stacker input without changing the primary value. □

The transition count excludes witness reconstruction and sorting overhead. We restrict the evaluated solver to at most 10^6 states, and the exactness guarantee applies to every instance processed within this limit.

D. Secondary Objective and Local Refinement

Exact primary coverage restricts the comparison to feasible sets that attain $W^\star$, but it does not make the secondary problem globally tractable. We first explain why the two objectives cannot be replaced by one universal weighted sum and then state the guarantee of the deployed local refinement.

Let $\mathcal{V} = \{C_{\mathrm{sem}}(S) : S \in \mathcal{F}_B\}$ contain the achievable primary values. When $\mathcal{V}$ has at least two values, let Δ_{inst} be its smallest positive gap. Because $0 \le \Phi(S) \le 1$, any coefficient $K > 1/\Delta_{\mathrm{inst}}$ makes every maximizer of $KC_{\mathrm{sem}} + \Phi$ lexicographic for that fixed instance. Normalized class weights can make the smallest positive coverage gap arbitrarily small, so no universal finite coefficient is available. The strict order in Eq. 8 avoids this instance-specific scale tuning.

Proposition V.4 (Secondary structure)**.** For fixed calibration statistics, U, C_{pred}, and D_{rep} are normalized, monotone, submodular set functions. Consequently, their nonnegative weighted combination Φ has the same properties.

Proof. For U and C_{pred}, each summand applies the nondecreasing concave function h_τ to a nonnegative modular sum. Therefore, the gain from adding the same candidate cannot increase as the current set grows. For D_{rep}, target k contributes $\max_{i\in S} \kappa_{i,k}$, whose marginal gain is

$$\max\{0, \kappa_{a,k} - \max_{i\in S} \kappa_{i,k}\}.$$

As S grows, this gain also cannot increase. Averaging and taking a nonnegative weighted sum preserve all three properties. □

Submodularity provides useful structure for Φ, but it does not yield a global secondary approximation guarantee under the exact primary constraint. The equal primary family

$$\mathcal{F}^\star = \{S \in \mathcal{F}_B : C_{\mathrm{sem}}(S) = W^\star\} \tag{33}$$

is not downward closed and need not satisfy an exchange axiom. Standard monotone submodular knapsack guarantees, including $1 - 1/e$, therefore do not apply to the secondary refinement stage.

For $S \in \mathcal{F}^\star$, let $\mathcal{N}_1(S)$ contain every set obtained by removing exactly one selected candidate and adding exactly one unselected candidate while preserving budget feasibility and $W^\star$.

Theorem V.5 (Coverage preservation and one swap certificate)**.** Given an exact primary solution, the refinement procedure terminates after finitely many accepted operations and returns $\widehat{S} \in \mathcal{F}^\star$ such that

$$\Phi(S') \le \Phi(\widehat{S}) + \varepsilon_{\mathrm{loc}}, \qquad \forall S' \in \mathcal{N}_1(\widehat{S}). \tag{34}$$

Proof. The constrained seed has coverage $W^\star$. Because C_{sem} is monotone and $W^\star$ is already maximal, every feasible fill keeps that value. The algorithm checks every accepted replacement for budget feasibility and equality to $W^\star$. Each accepted operation also increases Φ by more than $\varepsilon_{\mathrm{loc}}$. Because the candidate pool has only finitely many subsets, the procedure terminates. At termination, the exhaustive single swap search has found no feasible coverage neutral replacement whose gain exceeds the tolerance, which proves Eq. 34. □

The certificate is deliberately local because the refinement procedure explores only one swap neighbors. During fill, the procedure ranks candidates by gain per byte and stops when the chosen candidate's absolute gain does not exceed the tolerance. The guarantee does not cover broader additions, multi candidate exchanges, or the globally optimal secondary value.

E. Effective Coverage under Client Mixture Shift

Exact primary coverage preserves declared source support, but declared support alone does not certify current competence. We now connect these quantities for the pretrained returned set and quantify how client mixture shift changes that connection.

The certificate reuses statistics already computed during calibration and does not alter the selection objective.

Let $\vartheta_c \in [0, 1]$ be a prespecified current competence threshold. To test whether declared source support is backed by current scores, define

$$C_{\mathrm{cert}}(S; \boldsymbol{\vartheta}) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1 \wedge a_{i,c} \ge \vartheta_c\}. \tag{35}$$

Under this rule, a class contributes only when a selected detector provides both declared source support and a conservative current utility score. For comparison, let $C^a_\mu(S; \boldsymbol{\vartheta})$ replace $a_{i,c} \ge \vartheta_c$ by $\mu^a_{i,c} \ge \vartheta_c$, where $a \in \{\mathrm{cal}, \mathrm{dep}\}$.

Let $h = (j, c)$ index a client–class cell. Suppose that detector behavior within each cell remains stable between calibration and deployment while only the mixture weights change. Let $R_{i,h} \in [0, 1]$ denote the cell risk, and let π^{cal}_h and π^{dep}_h denote the two mixtures. For $a \in \{\mathrm{cal}, \mathrm{dep}\}$, define

$$R^a_i = \sum_{j,c} \pi^a_{j,c} R_{i,j,c}, \qquad \mu^a_{i,c} = \sum_j \pi^a_{j|c}(1 - R_{i,j,c}), \tag{36}$$

and let $\mathrm{TV}(p, q) = \frac{1}{2}\|p - q\|_1$. Under the calibration design, the means in Eqs. 26–27 equal $1 - R^{\mathrm{cal}}_i$ and $\mu^{\mathrm{cal}}_{i,c}$, respectively.

Theorem V.6 (Effective coverage under mixture shift)**.** Suppose that the within class client mixtures differ by at most η_c. On the simultaneous calibration event, every adaptively selected set $\widehat{S}$ satisfies

$$\begin{aligned} C^{\mathrm{cal}}_\mu(\widehat{S}; \boldsymbol{\vartheta} + \boldsymbol{\eta} + 2\boldsymbol{r}) &\le C_{\mathrm{cert}}(\widehat{S}; \boldsymbol{\vartheta} + \boldsymbol{\eta}) \\ &\le C^{\mathrm{dep}}_\mu(\widehat{S}; \boldsymbol{\vartheta}) \le C_{\mathrm{sem}}(\widehat{S}). \end{aligned} \tag{37}$$

where $(\boldsymbol{r})_c = r_c$ and $(\boldsymbol{\eta})_c = \eta_c$.

Moreover, if $\mathrm{TV}(\pi^{\mathrm{dep}}, \pi^{\mathrm{cal}}) \le \eta$, then

$$R^{\mathrm{dep}}_i \le R^{\mathrm{cal}}_i + \eta. \tag{38}$$

Proof. Theorem V.2 and the mixture assumption give

$$\mu^{\mathrm{cal}}_{i,c} \ge \vartheta_c + \eta_c + 2r_c \Longrightarrow a_{i,c} \ge \vartheta_c + \eta_c \Longrightarrow \mu^{\mathrm{dep}}_{i,c} \ge \vartheta_c.$$

Intersecting these implications with the fixed support edge $\Gamma_{i,c} = 1$, taking the existential quantifier over selected candidates, and summing class weights proves Eq. 37.

For the final claim, total variation bounds the change in the expectation of any $[0, 1]$-valued function. Applying this bound to the cell risks yields Eq. 38. □

By requiring both declared source side support and current class conditional Brier utility, C_{cert} connects the two under client mixture shift. During selection, SILOSTITCH optimizes C_{sem} and locally refines Φ while preserving the exact primary value. Only after selection do C_{cert} and the mixture sensitivity bound analyze the pretrained returned set.

VI. EVALUATION

We evaluate whether SILOSTITCH can reduce the cost of integrating pretrained detectors while preserving the source side class support needed to represent infrequent attacks. We organize the evaluation around five questions:

- **RQ1: Detection quality under a budget.** How much predictive quality and source support does the shared detector retain under a hard selection cost budget?
- **RQ2: Client label heterogeneity.** How do different client label distributions affect partition feasibility, class support, and detection quality?
- **RQ3: Selection signals and learners.** Which selection signals, semantic settings, and learning backends materially change the result?
- **RQ4: Score randomization.** What detection and attack leakage costs accompany stronger Laplace score randomization?
- **RQ5: Scalability and runtime.** Does exact selection remain correct and practical as the candidate pool and class vocabulary grow, and what logical communication saving does it deliver at the evaluated deployment scale?

A. Experimental Setup

Datasets. We use four multiclass cybersecurity datasets: CIC DDoS2019 (DDoS) [26], CICMalDroid2020 (MalDroid) [27], CIC Darknet2020 (Darknet) [28], and CIRA CIC DoHBrw 2020 (DoH) [29]. We use 66 clients in each task and let every client contribute one candidate detector. To keep all 66 candidates trainable, we give each client between 74 and 30,517 records for local detector fitting and at least two classes. Holding the federation size constant prevents cross dataset differences from being attributed merely to a larger or smaller candidate pool. The central 50% operating point limits the selected set to approximately half of the full pool selection cost. If all candidate costs were equal, this budget would correspond to roughly 33 of the 66 detectors, but the actual set size follows the logical byte costs. In RQ5, we vary the candidate pool M from 16 to 256 independently to test how selection scales. Table II summarizes the evaluation scale. Among the 13 observed classes in the reserved DDoS test set, one attack class is absent from client training. This class contributes to macro F1 but not to minimum evaluated class recall.

Protocol. We divide each training pool into disjoint 60%, 10%, and 30% partitions for local detector training, selection validation, and shared stacker training. After this split, the coordinator holds the 10% selection validation partition. We keep the test set sealed until both selection and stacker fitting have finished. Unless stated otherwise, we use five repeated splits, LightGBM [30] for local detectors and the shared stacker, a 50% selection cost budget, minimum source support $n_{\min} = 2$, class weights that are inversely proportional to the square root of class frequency, and equal weights for the three secondary selection signals.

Baselines and reference point. We compare SILOSTITCH with two baselines under the same selection cost budget. For the random baseline, we generate a random candidate order from each of 20 fixed selection seeds within a data split. The coordinator scans each order once and adds a detector whenever it fits the remaining budget, after which we average the 20 selections within the split. Under the individual utility baseline, the coordinator ranks detectors by mean calibrated Brier utility per selection cost byte and greedily packs the fixed order. Random selection tests whether an arbitrary budget feasible subset is sufficient, whereas individual utility ranking tests whether prioritizing current average behavior sacrifices source side class support. We use the full pool of 66 detectors as the unconstrained reference and the 100% selection cost reference point. By contrast, SILOSTITCH first maximizes weighted source class support and then refines current predictive quality without reducing that support. We later remove only the source support priority while retaining the same secondary search.

Metrics and statistics. We choose macro F1 as the main detection score because it gives every class equal weight. In imbalanced security traffic, this prevents a frequent class from hiding weak detection of a rarer attack class. We also report accuracy for overall correctness, balanced accuracy for average class recall, and minimum evaluated class recall. Among labels that appear in the sealed test set and belong to the fixed training class set, the last metric takes the lowest recall. We measure weighted class coverage separately because it describes retained training support rather than classifier quality. For the DDoS privacy experiment, we report *attack leakage*, which is the fraction of predictions labeled benign that are actually attacks. We define the selection cost in Eq. 5 and the budget in Eq. 6. For system results, we add the initial candidate upload and stacker label upload according to Eq. 7. We report means and 95% Student t confidence intervals over five repeated splits unless stated otherwise. When two conditions share a split, we pair their observations. We apply Holm correction within each comparison family that we fixed before analysis. We recompute every detection score from the reserved test confusion matrix.

Experimental environment. We run a centralized simulation on one machine under Linux and Python 3.9.25. In this simulation, we use NumPy 1.26.4, scikit-learn 1.3.2, Light-

Table II
SCALE OF THE FOUR EVALUATION TASKS; EACH TASK USES 66 CLIENTS AND ONE CANDIDATE DETECTOR PER CLIENT, EVALUATED CLASSES COUNT LABELS IN THE SEALED TEST SET, AND DDOS INCLUDES ONE TEST CLASS ABSENT FROM CLIENT TRAINING.

Dataset	Training samples	Test samples	Features	Evaluated classes
DDoS	3,355,966	1,127,526	79	13
MalDroid	8,118	3,480	470	5
Darknet	99,066	42,459	76	11
DoH	1,005,708	431,034	29	4

GBM 4.6.0, and XGBoost [31] 1.4.2. We record wall clock time and the logical bytes exchanged by the evaluated protocol.

B. RQ1: Detection Quality under a Budget

RQ1 asks whether SILOSTITCH can convert a hard cost budget into deployment savings without discarding either current detection quality or available source support. We first compare the methods at the central 50% operating point and then inspect weak class behavior and the full budget sweep.

Overall detection quality. At the 50% selection cost budget, SILOSTITCH uses 49.5% to 50.0% of the full pool selection cost. Its absolute mean macro F1 gap from the full pool is at most 0.0052 on MalDroid, Darknet, and DoH, whereas the gap reaches 0.0258 on DDoS. Thus, the same operating point closely tracks full pool mean quality on three tasks while exposing DDoS as the principal quality limit. Across SILOSTITCH's budget sweep in Fig. 2, MalDroid shows the clearest improvement and DoH rises only slightly, whereas DDoS and Darknet are non monotonic. Additional budget therefore does not universally guarantee higher test macro F1 in these experiments.

Weak class behavior and source support. Across MalDroid, Darknet, and DoH, the selected set at the 50% selection cost budget changes accuracy by at most 0.0046 and minimum evaluated class recall by at most 0.0061 relative to the full pool. On DDoS, SILOSTITCH loses 0.0296 accuracy but only 0.0094 minimum evaluated class recall. On Darknet, even the full pool reaches only 0.0077 minimum evaluated class recall, compared with 0.0058 after selection. The low Darknet recall is therefore visible in both configurations rather than being introduced by budgeted selection. More importantly, SILOSTITCH reaches the maximum weighted source support available under the budget on all four datasets.

Budget matched baselines. Relative to random selection, SILOSTITCH has a higher mean macro F1 on DDoS, MalDroid, and Darknet, while the DoH means differ by 0.0010. Relative to individual utility ranking, SILOSTITCH has a higher mean on MalDroid and Darknet and differs by 0.0005 on DoH. DDoS reveals the intended exception: individual utility ranking gains 0.0150 macro F1 at the 50% selection cost budget but retains only 0.910 of the available weighted source support, whereas SILOSTITCH attains the full support value of 1.0. This comparison makes SILOSTITCH's advantage explicit: it preserves the declared source coverage before refining current utility rather than claiming universal test score dominance.

Takeaway. At the 50% selection cost budget, SILOSTITCH attains the maximum weighted source support available under that budget and remains close to the full pool mean quality on three of four tasks, while DDoS reveals the deliberate support–utility tradeoff.

C. RQ2: Client Label Heterogeneity

RQ2 separates two consequences of label heterogeneity: whether a valid multiclass federation can be constructed and, conditional on feasibility, whether detection quality degrades. We evaluate both naturally occurring and controlled label distributions. To preserve the original deployment structure, we first keep the original DDoS and Darknet client assignments and repeat only the data split. To isolate label imbalance, we then draw client partitions with Dirichlet parameters $\alpha \in \{10, 1, 0.5, 0.1\}$ and compare them with an independent and identically distributed allocation. We require every accepted partition to provide at least two local training classes to each client so that each local detector remains a multiclass model. Figure 3 separates partition feasibility from detection quality as client label imbalance increases.

Partition feasibility. Darknet remains feasible for all ten seeds at every evaluated level, even as measured divergence rises from 0.0008 under the balanced allocation to 0.3516 at $\alpha = 0.1$. For MalDroid, all ten balanced and all ten $\alpha = 10$ partitions are usable, but only five $\alpha = 1$ partitions satisfy the same rule. At $\alpha = 0.5$ and 0.1, none of the ten fixed seeds produces a valid partition of the five class training pool across 66 clients while preserving at least two classes per client, despite allowing up to 100 generation attempts per seed. In these settings, partition feasibility becomes a limiting factor before the selector can be evaluated.

Quality within feasible partitions. Conditional on a usable partition, neither dataset shows a monotonic macro F1 decline as heterogeneity increases. Darknet remains between 0.5895 and 0.5976 at the 50% selection cost budget, while MalDroid remains between 0.8328 and 0.8375 over its usable range. These narrow descriptive ranges indicate that feasibility, rather than a consistent loss of aggregate quality, is the more visible effect of stronger heterogeneity in the controlled experiment.

Original client layouts. Under the original assignments, additional budget is reflected more strongly in the weakest class metric. As the budget rises from 25% to 75%, Darknet macro F1 increases from 0.6162 to 0.6290, while minimum evaluated class recall increases from 0.0452 to 0.0964. Over the same range, DDoS macro F1 changes from 0.6039 to

Table III

MACRO F1 AT THE 50% SELECTION-COST BUDGET; RANDOM SELECTION AND INDIVIDUAL UTILITY RANKING RECEIVE THE SAME BUDGET, THE FULL DETECTOR POOL IS AN UNCONSTRAINED REFERENCE, AND ENTRIES REPORT MEANS AND 95% STUDENT t CONFIDENCE INTERVALS OVER FIVE REPEATED SPLITS.

Method	DDoS	MalDroid	Darknet	DoH
Full detector pool	0.6412 ± 0.0121	0.8449 ± 0.0123	0.5972 ± 0.0128	0.6073 ± 0.0042
Random selection	0.6079 ± 0.0287	0.8344 ± 0.0100	0.5931 ± 0.0071	0.6074 ± 0.0028
Individual utility ranking	0.6303 ± 0.0437	0.8374 ± 0.0132	0.5921 ± 0.0168	0.6069 ± 0.0042
SILOSTITCH	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038

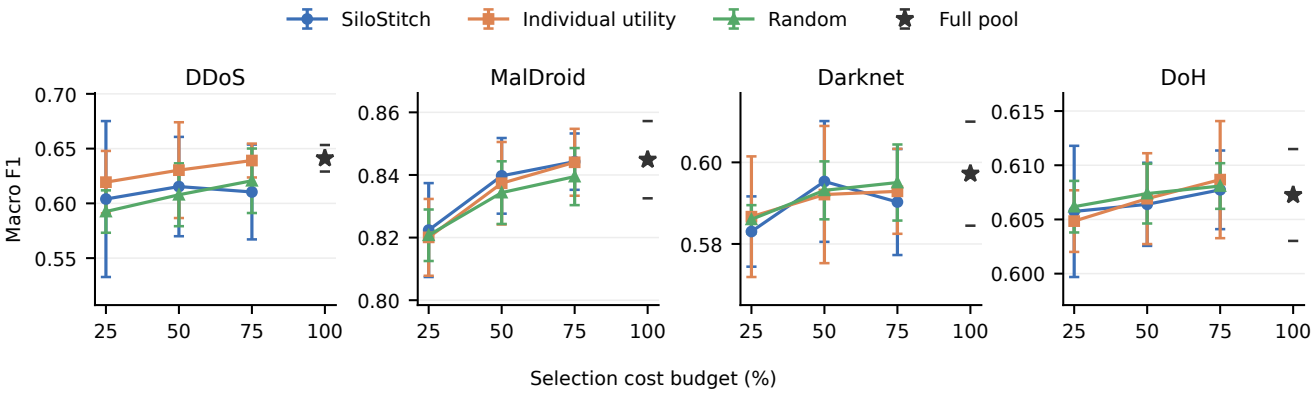

Figure 2. Macro F1 as a function of the selection cost budget. Curves compare SILOSTITCH with two budget matched selection rules at 25%, 50%, and 75%, and the star is the unconstrained pool of 66 detectors. Error bars are 95% confidence intervals over five repeated splits, after averaging the 20 random selections within each split. Each dataset uses its own vertical scale.

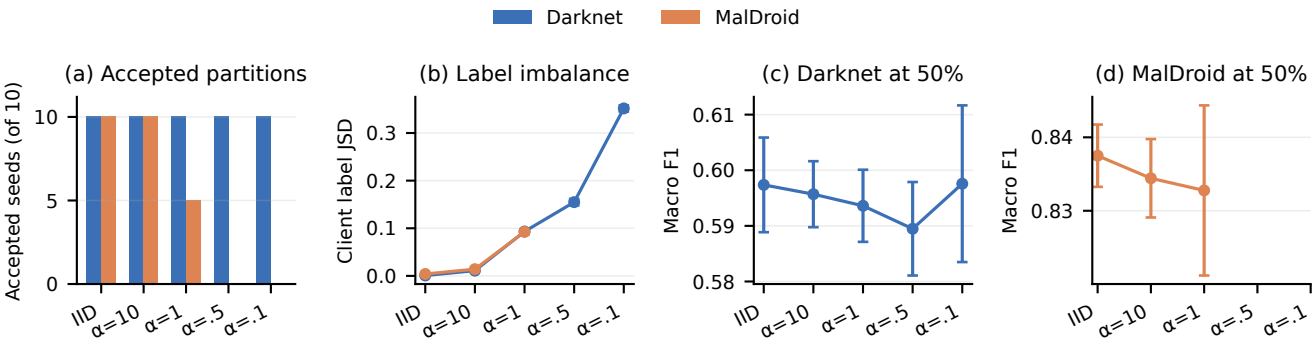

Figure 3. Effect of client label heterogeneity on partition feasibility and macro F1 at the 50% selection cost budget, where smaller Dirichlet α denotes stronger imbalance. (a) counts valid partitions among ten fixed seeds. (b) reports Jensen–Shannon divergence. (c) and (d) report macro F1. Error bars are 95% confidence intervals over valid seeds, with $n = 10$ except for MalDroid at $\alpha = 1$, where $n = 5$. The reason of missing MalDroid points in (b) is that no seed satisfies the two class per client requirement.

0.6104, while minimum evaluated class recall changes from 0.0568 to 0.0773. The extra budget is therefore reflected more strongly in the worst observed class than in the aggregate score for these two layouts.

Support priority ablation. We remove only the source support priority while retaining the same secondary search. Across 30 comparisons with the original assignments and 225 comparisons with controlled assignments, both versions choose the same set and obtain the same macro F1, minimum evaluated class recall, communication, and attained support. The support constraint is therefore not binding in these evaluated layouts because the secondary refinement already reaches the maximum available support. This result does not show that source priority is redundant for arbitrary candidate pools.

Takeaway. Within feasible partitions, controlled heterogeneity has limited descriptive impact on macro F1, while the smaller MalDroid task reveals a clear partition feasibility limit and the evaluated support constraint remains not binding.

Table IV
MACRO F1 AT THE 50% SELECTION-COST BUDGET FOR EACH LOCAL-TO-SHARED LEARNER PAIR; ENTRIES ARE MEANS OVER FIVE SPLITS, AND BOLD MARKS THE LARGEST DESCRIPTIVE MEAN FOR EACH DATASET.

Dataset	XGB →XGB	LGBM →XGB	XGB →LGBM	LGBM →LGBM
DDoS	0.617	**0.635**	0.633	0.615
MalDroid	0.806	0.833	0.827	**0.840**
Darknet	0.578	0.585	0.592	**0.595**
DoH	0.592	0.591	**0.607**	0.606

D. RQ3: Selection Signals and Learners

RQ3 distinguishes selector level choices from learner level choices. We first ablate the three secondary signals, then vary the support semantics, and finally compare local and shared learning backends.

Secondary selection signals. After the primary objective fixes maximum class support, SILOSTITCH uses record level utility, conservative classwise scores, and behavioral representativeness to order the remaining feasible sets. We remove these three signals one at a time while every variant still attains the maximum support. At the 50% selection cost budget, removing record level utility lowers DDoS macro F1 by 0.0190, whereas removing behavioral representativeness raises it by 0.0181. These effects change direction across datasets and budgets, and no removal produces a statistically significant change after Holm correction. No single signal therefore provides an independent, universal improvement in the evaluated settings. Instead, the signals collectively order candidate sets that already attain the same maximum support.

Semantic settings. We vary $n_{\min} \in \{1, 2, 5, 10\}$ and replace inverse square root class weights with uniform weights. Across two fixed Darknet layouts and two learner combinations, these choices leave the selected set, support, macro F1, and minimum evaluated class recall unchanged. These settings provide no tuning benefit in the tested layouts, although the result should not be generalized beyond them. By contrast, the learning backends produce visible score differences, as Table IV shows.

Learning backends. On DDoS, LightGBM local detectors with an XGBoost shared learner obtain the largest descriptive mean. On MalDroid and Darknet, LightGBM obtains the largest mean at both levels. On DoH, XGBoost local detectors with a LightGBM shared learner lead by a small margin. After Holm correction, only the two matched DoH comparisons between shared learners remain statistically significant. In both, the XGBoost shared learner is lower than its LightGBM alternative. Backend choice causes more visible numerical variation than the tested semantic settings, but the corrected statistical results are limited to DoH.

Takeaway. The evaluated selection signals and semantic settings do not yield a universal independent gain, whereas learning backend choice is the more actionable tuning dimension, with corrected results limited to DoH.

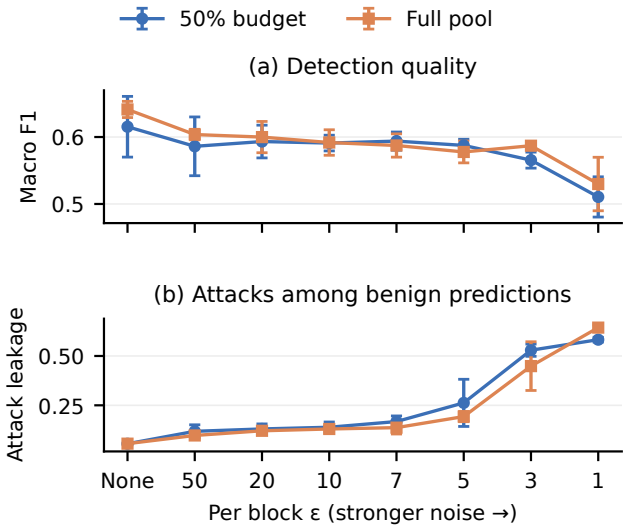

Figure 4. Effect of Laplace score randomization on DDoS. Panel (a) reports macro F1, and panel (b) reports the fraction of predictions labeled benign that are attacks. From left to right, per block ϵ decreases and noise strength increases. Curves compare SILOSTITCH at the 50% selection cost budget with the full pool, and error bars are 95% confidence intervals over the same five splits. The displayed ϵ is per released block, before composition.

E. RQ4: Score Randomization

RQ4 quantifies the detection cost of a formally private score release rather than estimating empirical attacker success. We freeze the selected set and add independent Laplace noise with scale $2/\epsilon$ to every score block used for shared stacker training. As ϵ decreases, the noise grows stronger. Here, ϵ is the privacy parameter for one released score block. Under the public label neighboring relation in Theorem V.1, two neighboring records share a label and differ in one feature vector. Under this relation, each released block satisfies record level local differential privacy conditional on the public label. When one record contributes s released blocks, sequential composition gives a total parameter of $s\epsilon$ for that upload. During the centralized sensitivity experiment, we sample the same noise distribution after selection and keep the test encoding fixed so that the curves isolate the effect on stacker training.

Aggregate detection quality. Without added noise, the selected set at the 50% selection cost budget and the full pool obtain macro F1 scores of 0.6153 and 0.6412. At the strongest evaluated setting, $\epsilon = 1$, the scores fall to 0.5105 and 0.5298. Relative to their no noise settings, the selected set loses 0.1048 and the full pool loses 0.1114. The two configurations therefore exhibit similar absolute degradation under the strongest perturbation.

Operational attack leakage. Attack leakage reveals a sharper security cost than macro F1 alone. For the selected set at the 50% selection cost budget, it rises from 0.055 without noise to 0.262 at $\epsilon = 5$ and 0.583 at $\epsilon = 1$. For the full pool, it rises from 0.056 to 0.193 and then to 0.644. At $\epsilon = 1$, more than half of the predictions labeled benign are attacks under

both configurations. A moderate aggregate score can therefore conceal an operationally costly failure, motivating the joint report of macro F1 and attack leakage.

Takeaway. Stronger per block score protection imposes a visible detection cost and can make benign predictions operationally unreliable, while the selected set does not exhibit a larger absolute degradation than the full pool.

F. RQ5: Scalability and Runtime

RQ5 evaluates three system properties in order: exactness on enumerable instances, scaling under larger candidate and class dimensions, and the end to end communication tradeoff.

Exactness audit. We compare SILOSTITCH with exhaustive search at $C = 8$ and $M \in \{8, 10, 12, 16\}$. Across ten repetitions at every size, the solver matches the exhaustive optimum in all 40 comparisons. Across these sizes, its median time ranges from 37.70 to 118.72 ms, while every 95th percentile remains below 125 ms. This audit validates the implementation against an enumerable ground truth, but it does not by itself establish runtime at larger scales. We next vary the number of candidates and attack classes separately in Fig. 5.

Scaling behavior. In the independently sampled candidate sweep with 12 attack classes, SILOSTITCH selects from 66 candidates in a median of 1.99 seconds and from 256 candidates in 33.73 seconds. In the independent class sweep with 66 candidates, raising the attack vocabulary from 12 to 20 classes increases the median from 2.95 to 100.10 seconds. Within the evaluated range, runtime becomes especially sensitive as the class vocabulary approaches 20 classes, making the class dimension the more visible scaling pressure near the upper end of the experiment.

System level tradeoff. To connect runtime and communication savings with detection quality, Table V reports the combined result. For runtime, we include every evaluated rule and budget within one data split.

Communication–quality tradeoff. Across the four tasks, SILOSTITCH reduces total logical communication by between 49.3% and 50.4% while losing at most 0.0258 macro F1 relative to the full pool. Across MalDroid, Darknet, and DoH, the absolute mean loss remains at or below 0.0052. For DDoS, the larger quality change still preserves the maximum available weighted source support reported in RQ1. The practical advantage is therefore an approximately one half reduction in logical communication with near full pool mean quality on three tasks and an explicit support–utility tradeoff on DDoS.

Runtime composition. When one split evaluates every rule and budget, its mean duration ranges from 0.78 to 50.09 minutes. For MalDroid and Darknet, stacker training dominates the total runtime. By contrast, selection takes the largest share on DDoS. The main computation cost therefore depends on the task rather than on selection alone. At the scale of 66 candidates, the exact selector itself finishes in seconds.

Takeaway. At the evaluated deployment scale, exact selection is practical and nearly halves logical communication, while the rapid rise near 20 attack classes defines the main observed scalability limit.

VII. DISCUSSION AND CONCLUSION

In summary, SILOSTITCH enables post-training integration of authorized pretrained intrusion detectors when historical source records are no longer accessible. It aligns heterogeneous detector outputs, preserves the best available weighted source class coverage under a deployment budget, and then improves current predictive behavior without sacrificing that coverage. Across four cybersecurity datasets, SILOSTITCH approximately halves logical communication under a 50% selection-cost budget while maintaining detection performance close to the full detector pool on most tasks.

Coverage Priority and Optimization Guarantees. The lexicographic objective is appropriate when preserving source class coverage is a deployment requirement rather than a quantity that can be freely traded for average predictive quality. By optimizing weighted coverage before current predictive behavior, it prevents secondary gains from compensating for losses in priority classes without requiring instance-specific weighting coefficients. Importantly, source coverage indicates which attack classes were represented during training, but does not guarantee current competence after distribution shift; coverage and class-specific predictive utility should therefore be examined separately. Within the calibrated candidate pool, the dynamic program exactly solves the primary coverage problem, whereas the subsequent additions and replacements preserve this optimum but provide only local, rather than global, guarantees for the secondary objective. The calibration guarantees also rely on the stated mixture-shift assumptions, and deployment under larger distribution shifts requires renewed validation.

Deployment Benefits and Practical Limits. The reported communication savings measure logical bytes for model delivery and aligned score uploads, rather than distributed latency or protocol overhead. Under the 50% selection-cost budget, SILOSTITCH achieves maximum weighted source coverage and near-full-pool predictive quality on MalDroid, Darknet, and DoH, while the larger gap on DDoS shows that maximum source coverage does not necessarily imply full-pool predictive parity. The mixed effects of secondary signals across datasets further suggest that their benefit is task dependent rather than universal. Moreover, the observed runtime increase as the attack vocabulary grows identifies the number of classes as an important scaling factor for the exact coverage solver.

Privacy Scope and Utility Tradeoff. When enabled, score protection requires each client to perturb every selected score block with fresh Laplace noise before transmission. For a public record label, each perturbed block satisfies record-level, label-conditional local differential privacy, with privacy loss composing across multiple released blocks. This guarantee applies only to randomized score uploads and does not protect public labels or contributed detector parameters; likewise, the absence of historical records alone does not constitute a privacy guarantee. Experiments further show that stronger perturbation can reduce predictive utility and increase attack leakage, indicating that deployment should balance

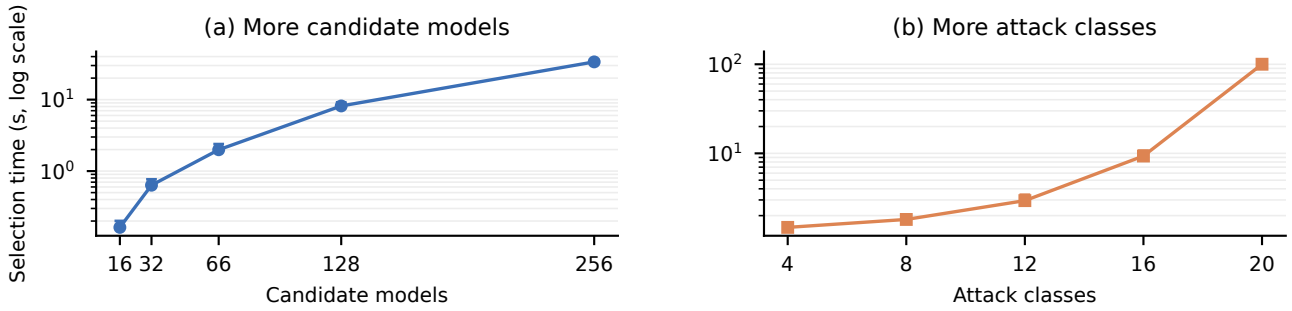

Figure 5. Exact selection time in two independently sampled sweeps. Panel (a) fixes 12 attack classes and varies the candidate pool, while panel (b) fixes 66 candidates and varies the attack class count. The vertical axis is logarithmic, and points are medians and upper bars are 95th percentiles over ten repetitions.

Table V
LOGICAL COMMUNICATION AND MACRO-F1 LOSS AT THE 50% SELECTION-COST BUDGET TOGETHER WITH CENTRALIZED REPLAY RUNTIME; RUNTIME IS THE MEAN TIME OVER FIVE SPLITS TO EVALUATE EVERY SELECTION RULE AND BUDGET ON ONE MACHINE, RATHER THAN ONE SELECTION CALL.

Dataset	Runtime (min)	Dominant stage	Full pool (MiB)	SILOSTITCH (MiB)	Communication saved	Macro F1 loss
DDoS	50.09	Selection (34.2%)	7,040.05	3,489.07	50.4%	0.0258
MalDroid	0.78	Stacker training (74.9%)	662.66	335.94	49.3%	0.0052
Darknet	42.51	Stacker training (89.1%)	3,436.78	1,732.29	49.6%	0.0019
DoH	8.54	Stacker training (33.2%)	2,942.76	1,488.51	49.4%	0.0009

privacy protection against both aggregate detection quality and security-critical class performance.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2. MLSys, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[3] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6357–6368. [Online]. Available: https://proceedings.mlr.press/v139/li21h.html

[4] H. Zhao, W. Du, F. Li, P. Li, and G. Liu, "Fedprompt: Communication-efficient and privacy-preserving prompt tuning in federated learning," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[5] C. S. Johnson, M. L. Badger, D. A. Waltermire, J. Snyder, and C. Skorupka, "Guide to cyber threat information sharing," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-150, 2016. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-150

[6] J. Xu, C. Li, Y. Zheng, and Z. Li, "ENTENTE: Cross-silo intrusion detection on network log graphs with federated learning," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s93-paper.pdf

[7] E. GDPR, "General data protection regulation (gdpr)," *Cit. on*, vol. 4, 2018.

[8] J. K.-W. Lee, P. Sattigeri, and G. W. Wornell, "Learning new tricks from old dogs: Multi-source transfer learning from pre-trained networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 4372–4382. [Online]. Available: https://papers.nips.cc/paper/8688-learning-new-tricks-from-old-dogs-multi-source-transfer-learning-from-pre-trained-networks

[9] Y. Wei, Z. Hu, L. Shen, Z. Wang, Y. Li, C. Yuan, and D. Tao, "Task groupings regularization: Data-free meta-learning with heterogeneous pre-trained models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 52 573–52 587. [Online]. Available: https://proceedings.mlr.press/v235/wei24e.html

[10] Q. Dong, A. Muhammad, F. Zhou, C. Xie, T. Hu, Y. Yang, S.-H. Bae, and Z. Li, "ZooD: Exploiting model zoo for out-of-distribution generalization," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/cd305fdee96836d5cc1de94577d71b61-Abstract-Conference.html

[11] X. Zhao, G. Sun, R. Cai, Y. Zhou, P. Li, P. Wang, B. Tan, Y. He, L. Chen, Y. Liang, B. Chen, B. Yuan, H. Wang, A. Li, Z. Wang, and T. Chen, "Model-GLUE: Democratized LLM scaling for a large model zoo in the wild," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=M5JW7O9vc7

[12] Y. Birman, S. Hindi, G. Katz, and A. Shabtai, "Cost-effective ensemble models selection using deep reinforcement learning," *Information Fusion*, vol. 77, pp. 133–148, 2022.

[13] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "CADE: Detecting and explaining concept drift samples for security applications," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2327–2344. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/yang-limin

[14] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[15] Information Commissioner's Office, "Guidance on AI and data protection," Mar. 2023, version 2.0.17. [Online]. Available: https://ico.org.uk/media/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/guidance-on-ai-and-data-protection-2-0.pdf

[16] S. Thirumuruganathan, M. Nabeel, E. Choo, I. Khalil, and T. Yu, "SIRAJ: A unified framework for aggregation of malicious entity detectors," in *43rd IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 507–521.

[17] T. Qi, F. Wu, C. Wu, L. He, Y. Huang, and X. Xie, "Differentially private knowledge transfer for federated learning," *Nature Communications*,

vol. 14, p. 3785, 2023. [Online]. Available: https://www.nature.com/articles/s41467-023-38794-x

[18] W. Qiu, Y. Feng, Y. Li, Y. Chang, K. Qian, B. Hu, Y. Yamamoto, and B. W. Schuller, "Fed-MStacking: Heterogeneous federated learning with stacking misaligned labels for abnormal heart sound detection," *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 9, pp. 5055–5066, Sep. 2024.

[19] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, "Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 4002–4015, 2023.

[20] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? a robust framework for detecting encrypted malicious network traffic," in *Network and Distributed System Security Symposium*, 2024. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-81-paper.pdf

[21] R. Xie, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, and M. Zhang, "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, 2023, pp. 625–642. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/xie

[22] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, "CoLD: Collaborative label denoising framework for network intrusion detection," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s1950-paper.pdf

[23] S. Maddock, G. Cormode, T. Wang, C. Maple, and S. Jha, "Federated boosted decision trees with differential privacy," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2249–2263. [Online]. Available: https://doi.org/10.1145/3548606.3560687

[24] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, 2013, pp. 429–438.

[25] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*. Springer, 2006, pp. 265–284.

[26] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.

[27] S. Mahdavifar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, "Dynamic android malware category classification using semi-supervised deep learning," in *IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 515–522.

[28] A. Habibi Lashkari, G. Kaur, and A. Rahali, "Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning," in *2020 the 10th International Conference on Communication and Network Security*, 2020, pp. 1–13.

[29] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of doh tunnels using time-series classification of encrypted traffic," in *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 63–70.

[30] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, pp. 3146–3154, 2017.

[31] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.